

Predicting Employee Attrition Using Cross-Validated Supervised and Unsupervised Machine Learning

Taufiq Mustafizur Rahman, ID - 24301485, Mohammad Ahonaf Bin Ameen, ID - 24301078,
Department of Computer Science and Engineering, BRAC University

Abstract—Predicting employee attrition is highly critical for modern human resource management. This project applies a rigorous machine learning pipeline to an imbalanced dataset of 1,677 employee records to identify factors driving attrition. We refined the dataset through extensive Exploratory Data Analysis (EDA) and pre-processing, utilizing Variance Inflation Factor (VIF), Pearson Correlation, and Spearman Rank Correlation to eliminate multicollinearity and non-linear noise. To resolve severe class imbalance and overcome the deceptive 88% Zero-R baseline, we implemented SMOTETomek resampling. We evaluated three supervised models:- Logistic Regression, K-Nearest Neighbors (KNN), and a Neural Network alongside an unsupervised K-Means clustering baseline. Utilizing a Stratified 5-Fold Cross-Validation approach, Logistic Regression achieved the highest F1 score (0.4442) and AUC score (0.7945) with an 84.37% accuracy. The Neural Network performed comparably well (85.27% accuracy, 0.7467 AUC), while the unsupervised K-Means algorithm struggled significantly (40.61% accuracy). This report details the methodology, feature engineering, training challenges, and aggregated cross-fold evaluation metrics.

Index Terms—Employee Attrition, Machine Learning, Stratified K-Fold, SMOTETomek, Neural Network, Logistic Regression, KNN, Classification, Hyperparameter Optimization.

I. INTRODUCTION

EMPLOYEE attrition represents a significant financial and operational challenge for organizations. Replacing an experienced employee incurs recruitment costs, training overhead, and an interim loss of productivity. By leveraging machine learning on historical HR data encompassing demographics, job roles, salary metrics, and satisfaction scores we aim to accurately predict the discrete binary target of "Attrition" (0 for stayed, 1 for left).

Because attrition events represent a minority of the workforce, building robust predictive models requires navigating severe class imbalance. If models are evaluated purely on standard train/test splits, they may appear highly accurate by simply predicting the majority class. Therefore, this project implements a rigorous evaluation methodology using Stratified 5-Fold Cross-Validation combined with advanced pre-processing techniques like SMOTETomek to ensure reliable, real-world applicability that balances both precision and recall.

II. DATASET DESCRIPTION

The dataset utilized is an HR analytics dataset tracking various employee metrics over time.

Dataset Size: The complete dataset consists of 1,677 employee records with 35 original features. To ensure robust mathematical validation and reduce computational overhead, we optimized these down to 23 highly predictive features.

Class Distribution: The dataset exhibits a severe class imbalance. Out of 1,677 employees, 1,477 (88%) stayed with the company, while only 200 (12%) left. Because of this, a simplistic algorithm that simply guesses "Stayed" every single time would inherently be 88% accurate without doing any actual learning. In machine learning, this is known as the **Majority Class Baseline** or **Zero-R Baseline**. This demonstrates why overall accuracy alone cannot dictate model performance. Without proper intervention, models fall victim to the "Accuracy Paradox," achieving this high baseline accuracy while completely failing to identify a single actual attrition risk.

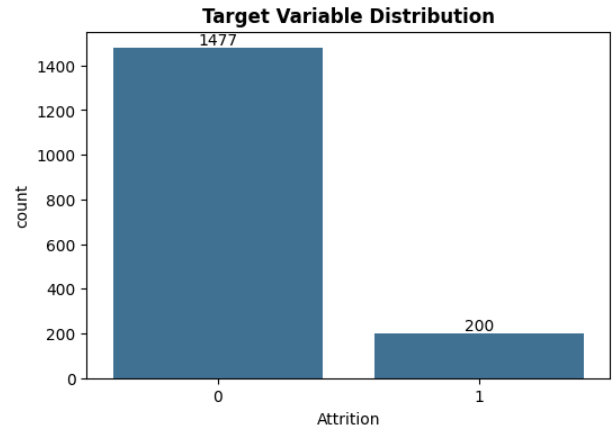


Fig. 1. Distribution of the target variable (Attrition), demonstrating a severe 88:12 class imbalance.

III. EXPLORATORY DATA ANALYSIS & PRE-PROCESSING

Thorough pre-processing is the foundation of any reliable machine learning pipeline. Failing to properly clean, scale, and balance data leads to artificial bias, runtime errors, and poor generalization on unseen data.

A. Handling Null and Duplicate Values

Raw datasets often contain missing data or duplicated rows. If left unaddressed, null values throw runtime exceptions during mathematical matrix operations, and duplicates cause

data leakage between training and testing sets, artificially inflating accuracy. We verified the dataset contained 0 nulls and 0 duplicate records, bypassing the need for statistical imputation.

B. Encoding Categorical Values

Machine learning models require numerical tensors to process data. If we skipped encoding, vital text-based categorical features (like "Department" or "BusinessTravel") would be discarded, resulting in the loss of critical predictive information. We applied `LabelEncoder` for binary/ordinal columns and `OneHotEncoder` (dropping the first column to avoid the dummy variable trap) to nominal categories, converting them into mathematically digestible vectors.

C. Handling Multicollinearity: Pearson Correlation and VIF

Including highly correlated features leads to multicollinearity, which causes parametric models like Logistic Regression to suffer from inflated coefficient variances, making them unstable. We generated a Pearson Correlation Heatmap (Fig. 2) to find linear relationships. The Pearson correlation coefficient (r) is defined as:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}} \quad (1)$$

To mathematically purge these redundant features, we calculated the Variance Inflation Factor (VIF). Features with a VIF score greater than 10.0 (e.g., `PerformanceRating`, `Age`, `JobLevel`) were iteratively removed to stabilize the model's weight distribution.

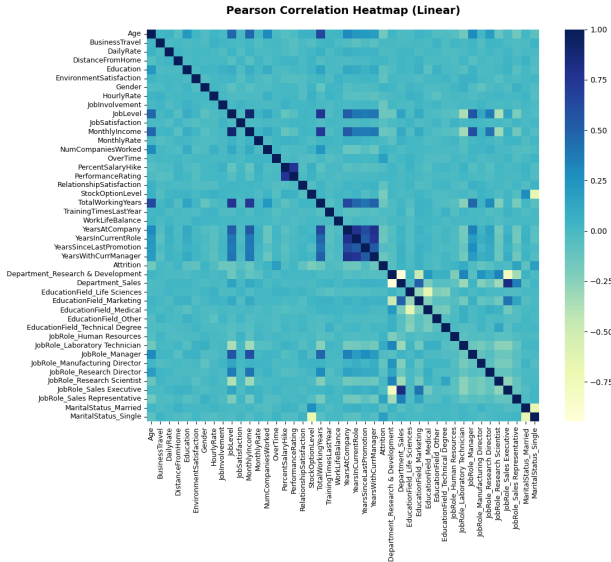


Fig. 2. Initial Pearson Correlation Heatmap highlighting linear relationships and multicollinearity among all 43 encoded features.

D. Non-linear Filtering: Spearman Rank Correlation

Pearson correlation only detects linear relationships. If we stopped there, ordinal features with non-linear relationships

would introduce unnecessary noise. We utilized Spearman Rank Correlation (ρ), which evaluates monotonic relationships based on ranked values (d_i being the difference in ranks):

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad (2)$$

Features with near-zero Spearman scores (-0.03 to +0.03), such as `BusinessTravel` and `DailyRate`, were dropped, resulting in a finalized, noise-free set of 23 features (Fig. 3).

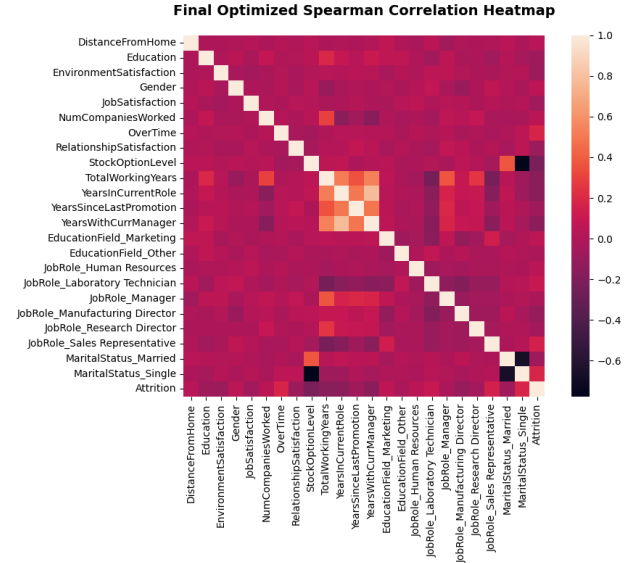


Fig. 3. Final Optimized Spearman Correlation Heatmap representing the surviving 23 predictive features.

E. Feature Scaling

Our optimized features spanned vastly different ranges (e.g., `MonthlyIncome` in the thousands vs. `Education` ranked 1 to 5). Without standardizing these values, distance-based algorithms (KNN) would be heavily skewed, treating large-magnitude features as more important. We applied a `StandardScaler` to force a mean (μ) of 0 and a standard deviation (σ) of 1:

$$z = \frac{x - \mu}{\sigma} \quad (3)$$

F. Class Imbalance: SMOTE and SMOTETomek

To solve the Accuracy Paradox and overcome the 88% Zero-R baseline, we utilized `SMOTETomek`. First, `SMOTE` (Synthetic Minority Over-sampling Technique) creates synthetic data points for the minority class by interpolating between nearest neighbors:

$$x_{\text{synthetic}} = x_i + \lambda(x_{\text{neighbor}} - x_i) \quad (4)$$

While `SMOTE` successfully balances the classes, it often generates synthetic points deep inside the majority class space, creating overlapping, noisy boundaries. `Tomek Links` solve this by finding pairs of sample data points from opposite classes that are also the nearest neighbors and then removing the majority class data points. This combination forces the models to learn clear, distinct decision boundaries rather than simply guessing the majority class.

IV. MODEL METHODOLOGIES AND TRAINING CHALLENGES

A. Cross-Validation Strategy: Stratified K-Fold

Problem faced: Relying on a single Train/Test split (e.g., an 80/20 split) results in performance metrics that are highly dependent on which specific data points randomly end up in the test set. Given our severe class imbalance, a single "lucky" or "unlucky" split could entirely misrepresent a model's true predictive capability.

Solution applied: We implemented **Stratified 5-Fold Cross-Validation**. The entire dataset was divided into 5 equal chunks (folds). The models were trained on 4 folds and tested on the 1 remaining fold, looping 5 times until every single employee record was evaluated as part of a test set exactly once. The "Stratified" parameter ensures that the exact 88:12 ratio of stayers to leavers was perfectly preserved in every fold. This guarantees our final metrics prove the average mathematical stability of the models across the entire population.

B. Neural Network

We constructed a multi-layer feed-forward Neural Network utilizing the Keras Sequential API (Fig. 4). The 23 input features map to a hidden layer of 64 neurons, followed by a second layer of 32 neurons, both utilizing the Rectified Linear Unit (ReLU) activation function:

$$\text{ReLU}(z) = \max(0, z) \quad (5)$$

The output layer utilizes a sigmoid activation to output a continuous attrition probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (6)$$

The loss function is binary cross-entropy, minimized using the Adam optimizer with a learning rate of 0.001 over 50 epochs with a batch size of 32:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (7)$$

Training Challenges: Neural networks are highly prone to overfitting on training data, especially when synthetic SMOTE data is introduced. To solve this, we inserted Dropout layers (rates of 0.3 and 0.2) to randomly disable neurons during training, forcing the network to learn generalized patterns rather than memorizing the training set.

Neural Network Architecture
Input (23) → Dense (64) → Dense (32) → Output (1)

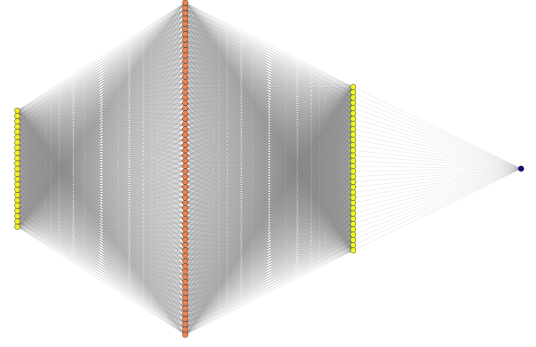


Fig. 4. The Neural Network Architecture mapped from input features through two hidden Dense layers to a single probabilistic output.

C. Logistic Regression

Logistic Regression estimates attrition probability by fitting a linear equation to the features and passing it through a logistic function:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w \cdot x + b)}} \quad (8)$$

Training Challenges: Logistic Regression fundamentally assumes independence among predictor variables. During early testing, multicollinearity caused erratic coefficient weights, making the model unreliable. This was entirely resolved by our rigorous VIF and Spearman purging phase. By supplying the 'class_weight='balanced'' parameter and training on SMOTETomek data, it became our most mathematically stable model.

D. K-Nearest Neighbors (KNN)

KNN predicts attrition by calculating the Euclidean distance between a test employee and historical training data:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (9)$$

Training Challenges: KNN suffers heavily from the "Curse of Dimensionality" that is, as the number of features increases, spatial distance calculations lose meaning. Our correlation filtering (reducing from 43 encoded features to 23) was critical for KNN to function. After testing varying K values, $K = 9$ provided the best balance between bias and variance on the balanced data.

E. K-Means Clustering (Unsupervised)

We used K-Means to group employees without feeding the algorithm the true "Attrition" labels, iteratively minimizing the variance within each cluster:

$$J = \sum_{j=1}^K \sum_{x \in C_j} \|x - \mu_j\|^2 \quad (10)$$

Training Challenges: K-Means completely failed to identify attrition risk. Without historical labels to guide it, the algorithm grouped employees by prominent numerical features like seniority and salary rather than their likelihood to leave. Because both junior and senior clusters were predominantly made up of people who stayed, mapping the clusters back to the binary target resulted in massive false positive misclassifications.

V. CROSS-VALIDATED RESULTS

A. Evaluation Metrics

Because accuracy is deceptive in imbalanced sets, we rely on Precision, Recall, and the F1-Score, calculated using True Positives (TP), False Positives (FP), and False Negatives (FN):

$$\text{Precision} = \frac{TP}{TP + FP} \quad (11)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (12)$$

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (13)$$

TABLE I
AGGREGATED 5-FOLD MODEL COMPARISON

Model	Accuracy	Precision	Recall	F1 Score	AUC
Logistic Regression	0.8437	0.4156	0.535	0.4442	0.7945
Neural Network	0.8527	0.4071	0.420	0.4000	0.7467
KNN	0.7448	0.2582	0.525	0.3379	0.7064
K-Means (Unsup.)	0.4061	0.1440	0.805	0.2443	0.5785

B. Confusion Matrices (Fold Averages)

Fig. 5 displays the average confusion matrix per validation fold (approx. 1677/5=335 samples per fold). In the context of our HR problem: **True Positives (TP)** are correctly identified attrition instances (leavers), **True Negatives (TN)** are correctly identified retention instances (stayers), **False Positives (FP)** are false alarms (stayers predicted to leave), and **False Negatives (FN)** are critical misses (leavers predicted to stay).

- **Logistic Regression:** The confusion matrix shows that the model achieved the best balance, correctly identifying 21.4 attrition instances (TP) and 261.6 retention instances (TN) per fold, while minimizing false alarms (33.8 FP) and critical misses (18.6 FN).
- **Neural Network:** The confusion matrix shows that the model caught 16.8 attrition instances (TP) and 269.2 retention instances (TN) per fold, while also maintaining a low false alarm rate (26.2 FP) however, suffering with slightly more critical misses (23.2 FN).
- **KNN:** The confusion matrix shows that the model identified 21 attrition instances (21.0 TP) and 228.8 retention instances (TN) correctly however, struggled heavily with precision, generating a massive number of false alarms i.e. 66.6 false alarms (FP) while missing 19.0 attrition instances (FN) per fold.
- **K-Means:** The confusion matrix shows that the model for unsupervised clustering aggressively overpredicted

attrition. While it flagged 32.2 attrition instances (TP), it also generated an abysmal 191.4 false alarms (FP) per fold. As a result, correctly identified retention instances (TN) and critical misses (FN) reduced to just 104.0 and 7.8 respectively per fold.

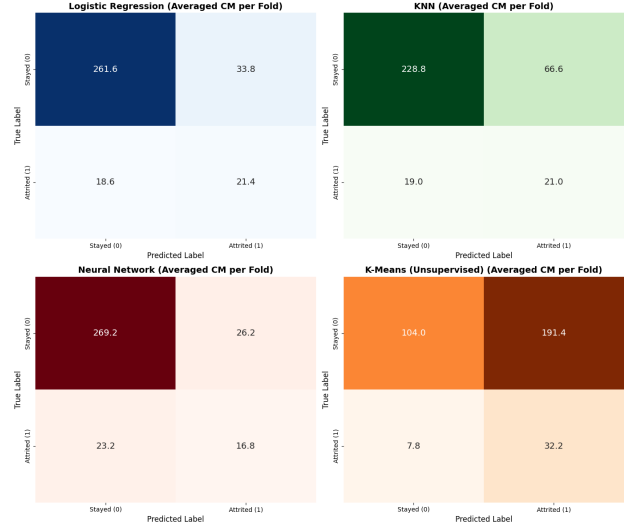


Fig. 5. Averaged Confusion Matrices across the 5 validation folds. Decimal values reflect the mathematical mean of TP, TN, FP, and FN counts per fold.

C. ROC Curves

The ROC curve evaluates the true positive rate versus the false positive rate across all prediction thresholds. Logistic Regression dominates with an aggregated AUC of 0.7945, followed by the Neural Network at 0.7467. KNN has an AUC score of 0.7064 and K-Means hugs the random-guess diagonal with a poor AUC of 0.5785.

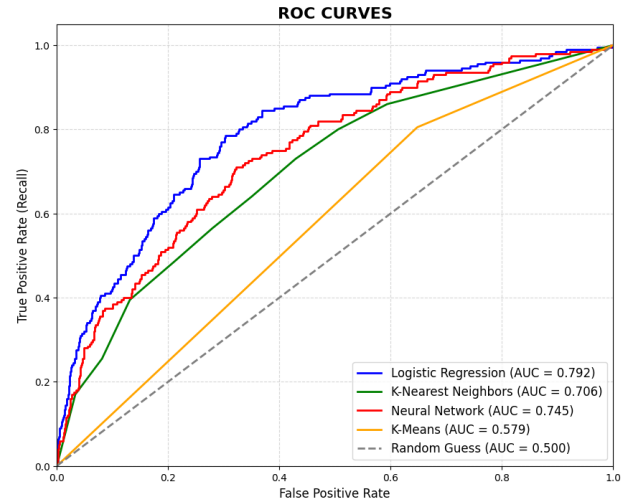


Fig. 6. Cross-validated ROC Curves displaying the superior class separation capabilities of Logistic Regression and the Neural Network.

VI. CONCLUSION

The project highlights the absolute necessity of exhaustive EDA and Cross-Validated evaluation when handling imbal-

anced HR datasets. Without techniques like VIF purging using Pearson correlation, optimized feature selection using Spearman correlation, and SMOTETomek resampling, models inevitably succumb to multicollinearity, the Curse of Dimensionality, and the Accuracy Paradox caused by the 88% Zero-R baseline.

In accordance with the confusion matrices and ROC curve, the Supervised models vastly outperformed the Unsupervised model. Logistic Regression emerged as the most mathematically stable and practical model, boasting the highest F1-score (0.44) and AUC value (0.79) alongside an 84% approximate accuracy. Its superior performance is primarily because, after our rigorous removal of multicollinearity, its linear probabilistic nature proved highly robust and less prone to memorizing synthetic noise. Similarly, the Keras Neural Network performed well with a comparatively high F1-Score (0.40) and AUC value (0.75) alongside a slightly higher 85% approximate accuracy. However, it slightly edged out in overall capability; its complex architecture made it more susceptible to overfitting on the synthetic SMOTETomek data, resulting in a higher rate of critical misses (FN). Next, K-Nearest Neighbors (KNN) struggled heavily due to the Curse of Dimensionality; even with reduced features, distance metrics become diluted, causing it to generate a massive rate of false alarms (FP). Lastly, the unsupervised K-Means clustering failed because it lacked historical target labels to guide its boundaries. It grouped employees based on arbitrary numerical similarities like seniority, essentially flagging half the company as an attrition risk.

In a real-world Human Resources context, the balance between Precision and Recall is paramount since a higher False Positive rate (i.e., poor precision) means the company wastes valuable retention budget and intervention time on employees who have no intention of leaving. Conversely, a higher False Negative rate (i.e., poor recall) means the company loses critical talent unexpectedly.

Therefore, higher F1-score and AUC values provide a reliable basis for selecting the optimal model for employee attrition prediction. Ultimately, the heavily optimized Logistic Regression model proved to be the most efficient, interpretable and balanced solution for targeted human resource analytics.

REFERENCES

- [1] Employee Attrition Dataset. Available: Dataset.csv.
- [2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [3] V. Jain and A. Nayyar, "Predicting Employee Attrition using Machine Learning," in *2018 International Conference on System Modeling & Advancement in Research Trends (SMART)*, Moradabad, India, 2018, pp. 1–7.
- [4] Y. Zhao, G. H. Homan, and J. C. K. Li, "Predicting Employee Turnover with Machine Learning Techniques," *IEEE Access*, vol. 8, pp. 125324–125333, 2020.